

Capitolo 13 - Interfaccia del file system

Status Done

Concetto di file

Il SO astrae le proprietà fisiche dei dispositivi di memoria (dischi, nastri, memorie flash) definendo un'unità logica di memorizzazione: il **file**.

- **Definizione:** un file è un insieme di informazioni correlate, registrate su memoria non volatile e identificate da un nome.
- **Tipologie:** i dati possono essere numerici, alfabetici o binari. Il significato del contenuto (bit, byte, righe o record) è definito dal suo creatore.
- **Esempi:** programmi sorgente, file eseguibili, documenti di testo, file multimediali (musica, video).

Attributi dei file

Ogni file è accompagnato da metadati che variano tra i SO:

- **Nome:** l'unica informazione leggibile dall'uomo.
- **Identificatore:** un numero unico (come l'inode in Unix) usato dal sistema.
- **Tipo:** necessario per i sistemi che supportano diverse estensioni.
- **Locazione:** puntatore alla posizione fisica sul dispositivo.
- **Dimensione:** grandezza attuale in byte o blocchi.
- **Protezione:** informazioni su chi può leggere, scrivere o eseguire.
- **Timestamp:** date di creazione, modifica e ultimo accesso.

Operazioni sui file

Il file è un **tipo di dato astratto**. Le operazioni fondamentali sono:

1. **Creazione:** ricerca dello spazio sul disco e inserimento nella directory.
2. **Scrittura:** richiede un **puntatore di scrittura** che indica dove inserire i prossimi dati.
3. **Lettura:** utilizza un **puntatore di lettura** (spesso lo stesso della scrittura per risparmiare risorse).
4. **Riposizionamento (Seek):** sposta il puntatore all'interno del file senza fare I/O.
5. **Cancellazione:** rilascia lo spazio sul disco e rimuove l'elemento dalla directory.
6. **Troncamento:** svuota il contenuto del file mantenendo però gli attributi e il nome.

Gestione dei file aperti

Per evitare di cercare ripetutamente nella directory, si usa la chiamata `open()`.

- **Tabella dei file aperti (Open-file table):** quando un file è aperto, i suoi attributi sono copiati in memoria.
- **Due livelli di tabelle:**
 - **Tabella per processo:** contiene informazioni specifiche (es. l'offset/puntatore corrente e i permessi).
 - **Tabella di sistema:** contiene informazioni indipendenti dal processo (es. posizione fisica sul disco e contatore delle aperture).
- **Contatore delle aperture (Open Count):** indica quanti processi usano il file; quando arriva a zero, il file può essere rimosso dalla memoria (chiusura effettiva).

Locking dei file

I lock proteggono i file da accessi concorrenti indesiderati:

- **Shared lock (condiviso):** come un lock di lettura; più processi possono leggere contemporaneamente.
- **Exclusive lock (esclusivo):** come un lock di scrittura; solo un processo può accedere.
- **Lock obbligatori (mandatory):** il SO impedisce l'accesso se il file è bloccato.
- **Lock consultivi (advisory):** il sistema non impedisce l'accesso; spetta ai programmatori scrivere codice che controlli lo stato del lock.

Tipi di file

Il SO utilizza spesso l'estensione per riconoscere il tipo di file e decidere come trattarlo.

Tipo di file	Estensione usuale	Funzione
Eseguibile	<code>.exe</code> , <code>.com</code> , <code>.bin</code>	Programma in linguaggio macchina pronto all'uso.
Sorgente	<code>.c</code> , <code>.java</code> , <code>.py</code>	Codice scritto da programmatori.
Archivio	<code>.zip</code> , <code>.tar</code> , <code>.rar</code>	Gruppo di file correlati, spesso compressi.
Multimediale	<code>.mp3</code> , <code>.mp4</code> , <code>.mov</code>	Dati audio o video.

- **Magic Number:** Unix inserisce un codice speciale all'inizio dei file binari per identificarne il formato, indipendentemente dall'estensione.

Struttura dei file

Alcuni sistemi impongono strutture rigide, altri (come Unix e Windows) considerano i file come semplici **sequenze di byte**.

- **Vantaggio della sequenza di byte:** massima flessibilità per le applicazioni.
- **Svantaggio:** il SO fornisce poco supporto; ogni applicazione deve saper interpretare i propri dati.

Struttura interna dei file

I dischi leggono e scrivono in **blocchi fisici** di dimensione fissa (es. 512 byte). Poiché i file hanno lunghezze variabili, il SO deve gestire l'**impaccamento**:

- Se un file non è multiplo esatto della dimensione del blocco, l'ultimo blocco sarà parzialmente vuoto.
- **Frammentazione interna:** lo spazio sprecato alla fine dell'ultimo blocco. Più i blocchi sono grandi, più spazio rischia di essere sprecato.

Metodi d'accesso

Per accedere alle informazioni contenute nei file è necessario trasferirli in memoria. Esistono molti metodi per accedere alle informazioni.

Accesso sequenziale

È il metodo più semplice e comune, ispirato al funzionamento dei nastri magnetici.

- **Funzionamento:** le informazioni vengono elaborate in ordine, una dopo l'altra.
- **Operazioni:**
 - `read_next()`: legge la porzione successiva e avanza il puntatore.
 - `write_next()`: aggiunge dati alla fine del file e sposta il puntatore alla nuova fine.
- **Uso:** è il metodo ideale per compilatori, editor di testo e riproduttori multimediali dove i dati servono in una sequenza logica continua.

Accesso diretto (accesso relativo)

Questo metodo si basa sul modello del disco e permette di accedere a qualsiasi blocco del file senza dover leggere quelli precedenti.

- **Struttura:** il file è visto come una sequenza di record di lunghezza fissa numerati.
- **Blocchi relativi:** l'utente utilizza un indice (0, 1, 2...) relativo all'inizio del file. Il SO si occupa poi di tradurre questo indice nell'indirizzo fisico assoluto sul disco.
- **Operazioni:** `read(n)` e `write(n)`, dove *n* è il numero del blocco desiderato.
- **Vantaggi:** fondamentale per i database e i sistemi di prenotazione (es. per leggere i dati del volo 713 basta accedere direttamente al blocco 713).
- **Simulazione:** è facile simulare l'accesso sequenziale su un file ad accesso diretto (basta incrementare un contatore di posizione), ma è quasi impossibile il contrario.

Altri metodi d'accesso

Partendo dall'accesso diretto, è possibile costruire strutture più complesse basate su un **indice**.

- **Indice (index):** è un file separato (o una parte del file) che contiene delle “chiavi” e dei puntatori. Per trovare un dato, il sistema cerca prima nell’indice e poi “salta” direttamente alla posizione indicata nel file principale.
- **Indici multi-livello:** se il file è enorme, anche l’indice diventa troppo grande per stare in memoria. In questo caso si crea un **indice degli indici**: l’indice primario punta a blocchi di un indice secondario, che a sua volta punta ai dati effettivi.
- **ISAM (Indexed Sequential Access Method):** tecnica classica (usata da IBM) per gestire grandi database. Permette di trovare un record tramite una chiave con pochissime letture su disco (solitamente due), combinando la velocità della ricerca binaria negli indici con l’efficienza dell’accesso diretto.

Struttura delle directory

La directory può essere vista come una “tabella dei simboli” che traduce il nome di un file nel suo corrispondente elemento informativo nel sistema.

Operazioni sulle directory

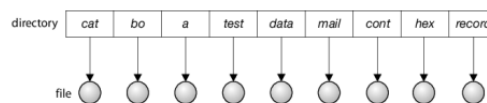
Per essere efficiente, una directory deve supportare le seguenti operazioni:

- **Ricerca:** trovare un file specifico o un gruppo di file (tramite espressioni regolari).
- **Creazione/Cancellazione:** aggiungere o rimuovere file.
- **Elencazione:** mostrare tutti i file contenuti e i loro attributi.
- **Ridenominazione:** cambiare nome al file (e talvolta spostarlo di posizione).
- **Attraversamento:** accedere a ogni directory per operazioni di manutenzione o backup.

Directory a un livello

È lo schema più semplice: tutti i file di tutti gli utenti sono in un’unica grande lista.

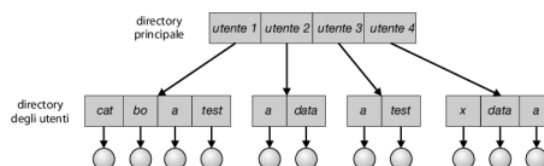
- **Vantaggi:** facilità di gestione.
- **Svantaggi:** **collisione dei nomi** (due utenti non possono usare lo stesso nome come `test.txt`) e difficoltà di organizzare quando il numero di file cresce.



Directory a due livelli

Ogni utente ha la propria directory personale (**UFD - User File Directory**). Esiste una directory principale (**MFD - Master File Directory**) che punta alle directory dei singoli utenti.

- **Vantaggi:** risolve il problema dei nomi duplicati tra utenti diversi.
- **Svantaggio:** isola gli utenti, rendendo difficile la cooperazione. Per accedere al file di un altro utente serve un **nome di percorso (path name)** come `/utenteB/prova.txt`.
- **Percorso di ricerca (search path):** se un comando non è nella directory corrente, il sistema lo cerca in directory predefinite (es. directory 0 per i file di sistema).

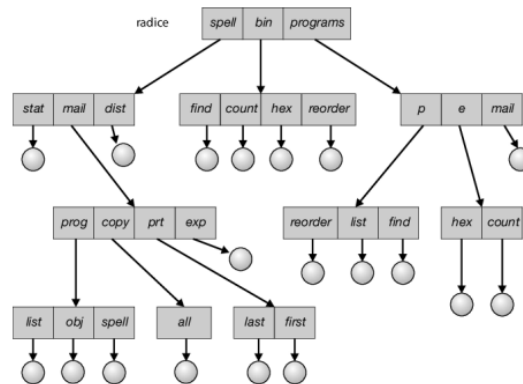


Directory con struttura ad albero

È l’evoluzione naturale: gli utenti possono creare sottodirectory per organizzare i propri file a piacere. È la struttura più comune nei sistemi moderni (Windows, Linux, MacOS).

- **Directory radice (root):** il punto di partenza dell’albero.
- **Directory corrente:** la posizione in cui l’utente si trova in un dato momento.
- **Nomi di percorso:**

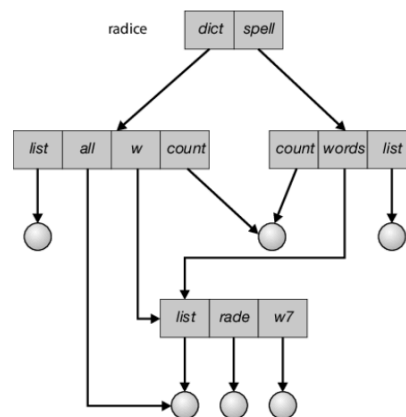
- **Assoluti**: partono dalla radice (es. `/spell/mail/prt/first`).
- **Relativi**: partono dalla directory corrente (es. `prt/first`).
- **Cancellazione**: alcuni sistemi permettono di cancellare solo directory vuote; altri (come `rm -r` in Unix) cancellano ricorsivamente tutto il contenuto.



Directory con struttura a grafo aciclico

Permette la condivisione di file o sottodirectory; uno stesso file può apparire in più directory contemporaneamente. Essendo "aciclico", non sono ammessi percorsi che ritornano su se stessi.

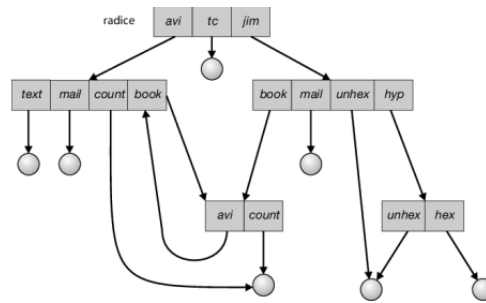
- **Collegamenti (Link)**:
 - **Link simbolici (Soft Link)**: puntatori indiretti (contengono il percorso di un altro file). Se il file originale viene rimosso, il link diventa "sospeso" (*dangling*).
 - **Link effettivi (Hard Link)**: il nome punta direttamente ai dati (inode). Il file viene eliminato fisicamente solo quando l'ultimo hard link viene rimosso (si usa un **contatore dei riferimenti**).



Directory con struttura a grafo generale

Se si permettono collegamenti arbitrari, possono crearsi dei cicli (una directory che contiene se stessa, direttamente o indirettamente).

- **Problemi**:
 - Gli algoritmi di ricerca potrebbero entrare in loop infiniti.
 - La cancellazione diventa complessa: un contatore di riferimenti non basta perché un ciclo di file potrebbe puntarsi a vicenda senza essere accessibile dall'esterno.
- **Soluzione**: è necessaria la **garbage collection** (un processo che attraversa tutto il sistema per trovare dati orfani), ma è un'operazione molto costosa. Per questo, molti sistemi limitano i collegamenti per mantenere la struttura aciclica.



Protezione

- **Affidabilità:** si riferisce alla salvaguardia dai danni fisici (guasti hardware, sbalzi di tensione, errori umani). Si ottiene principalmente tramite copie di backup periodiche e ridondanze (RAID).
- **Protezione:** si riferisce al controllo degli accessi impropri. In un sistema multiutente, è necessario stabilire chi può operare sui file e in che modo.

Tipi d'accesso

La protezione non è un interruttore "tutto o niente", ma un **accesso controllato**. Le operazioni che il sistema può monitorare e limitare sono:

- **Lettura (Read):** leggere il contenuto.
- **Scrittura (Write):** modificare o riscrivere il file.
- **Esecuzione (Execute):** caricare il programma in memoria ed eseguirlo
- **Aggiunta (Append):** scrivere nuovi dati solo alla fine del file.
- **Cancellazione (Delete):** rimuovere il file e liberare spazio.
- **Elencazione (List):** visualizzare nome e attributi dei file in una directory.

Controllo degli accessi

Il metodo più diffuso consiste nel far dipendere l'accesso dall'**identità dell'utente**.

Lista di controllo degli accessi (ACL)

Il sistema associa a ogni file/directory una lista (ACL) che specifica gli utenti e i relativi permessi.

- **Vantaggi:** massima precisione (posso dire esattamente cosa può fare ogni singola persona).
- **Svantaggi:** le liste possono diventare lunghissime e difficili da gestire in memoria.

Classificazione condensata (Proprietario, Gruppo, Universo)

Per semplificare, molti sistemi (specialmente Unix/Linux) dividono gli utenti in tre categorie:

1. **Proprietario (Owner):** chi ha creato il file.
2. **Gruppo (Group):** un insieme di utenti che collaborano (es. un team di progetto).
3. **Universo (Public/Other):** tutti gli altri utenti del sistema.

Unix questo si traduce in 9 bit di protezione (3 bit **rwx** per ogni categoria). Se un file ha permessi **rwxp-x---**, il proprietario può fare tutto, il gruppo può leggere ed eseguire, gli altri non possono fare nulla.

Sistemi ibridi (Solaris e Windows)

I sistemi moderni combinano i due approcci:

- **Solaris:** usa i permessi standard di default, ma permette di aggiungere una ACL specifica (segnalata con un "+") per casi particolari (es. dare accesso a un singolo ospite esterno al gruppo).
- **Windows:** gestisce le ACL principalmente tramite interfaccia grafica, permettendo di concedere o negare permessi specifici a utenti e gruppi in modo molto dettagliato.

Altri metodi di protezione

- **Password:** associare una parola d'ordine a ogni file o directory. È poco pratico se i file sono molti (troppe password da ricordare) e rischioso se se ne usa una sola per tutto.

- **Protezione delle directory:** le directory richiedono controlli specifici. Ad esempio, il bit di esecuzione su una directory in un Unix non significa “eseguire un programma”, ma “poter entrare nella directory” o attraversarla per raggiungere i file sottostanti.

Nota sulla precedenza: se i permessi del gruppo contrastano con quelli una ACL specifica, il sistema (come Solaris) applica il **principio di specificità:** la regola più precisa (quella della ACL) ha la priorità su quella generale.

File mappati in memoria

Un altro metodo per accedere ai file è unendo i concetti di file system e memoria virtuale: **file mappati in memoria** (*memory-mapped files*).

Meccanismo di base

Invece di usare le classiche chiamate `read()` e `write()`, che richiedono continui passaggi tra spazio utente e kernel (overhead), la mappatura in memoria permette di trattare i file come se fosse un semplice vettore di byte nella RAM.

- **Funzionamento:** il SO associa logicamente un blocco del disco a una o più pagine della memoria virtuale del processo.
- **Paginazione su richiesta:** inizialmente, l'accesso al file causa un *page fault*. Il SO carica quindi la porzione necessaria del disco alla memoria fisica. Ogni accesso successivo avviene alla velocità della RAM, senza chiamate di sistema.
- **Aggiornamento su disco:** le modifiche apportate in memoria non vengono scritte immediatamente sul disco (scrittura asincrona). Il file fisico viene aggiornato periodicamente o quando il file viene chiuso direttamente.
- **Esempio di Solaris:** in sistemi come Solaris, tutto l'I/O è trattato internamente come mappato in memoria. Se l'utente usa `read()`, il kernel mappa il file nel proprio spazio; se usa `nmap()`, lo mappa direttamente nello spazio utente.

Condivisione dei dati

Questa tecnica è il metodo d'elezione per implementare la **memoria condivisa** tra processi.

- Più processi mappano lo stesso file fisico nei rispettivi spazi di indirizzamento virtuali.
- Entrambi i processi puntano alla stessa pagina di memoria fisica.
- Una modifica effettuata dal processo A è istantaneamente visibile al processo B.
- È spesso supportata la tecnica **Copy-on-Write** (copiatura su scrittura) per permettere ai processi di condividere dati in sola lettura ma di creare copie private se decidono di modificarli.

Memoria condivisa nella API Windows

In Windows, la condivisione di memoria tramite file mappati segue un protocollo preciso basato su tre funzioni principali.

Il processo produttore (scrittura)

1. `CreateFile()` : apre il file fisico (es. `temp.txt`) e ottiene un riferimento (`HANDLE`).
2. `CreateFileMapping()` : crea un “oggetto di mappatura” con un nome specifico (es. `oggettoCondiviso`). Questo nome permette ad altri processi di trovare la stessa area di memoria.
3. `MapViewOfFile()` : mappa l'oggetto nello spazio degli indirizzi del processo e restituisce un puntatore. A questo punto, basta usare una funzione come `sprintf()` per scrivere dati direttamente in memoria.

Il processo consumatore (lettura)

Il consumatore non ha bisogno di conoscere il file fisico su disco, ma deve conoscere il nome dell'oggetto condiviso:

1. `OpenFileMapping()` : cerca l'oggetto chiamato `oggettoCondiviso` .
2. `MapViewOfFile()` : crea la propria vista dell'oggetto.
3. **Lettura:** usa un semplice `printf()` sul puntatore ottenuto per leggere il messaggio lasciato dal produttore.

Al termine, entrambi i processi usano `UnmapViewOfFile()` per scollegare la memoria e `CloseHandle()` per rilasciare le risorse.